3e7608b4-7459-4f05-8f0e-0576a18ae1

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw-firewall' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw-firewall/gh-aw-firewall (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-20T02:03:06.992+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[WIP] Refactor Docker startup retry tests to reduce code duplication
</pr_title>

<pr_description>
- [x] Create `src/test-helpers/startup-retry.test-utils.ts` with `mockStartupRetry()` and `expectComposeUpAttempts()` helpers
- [x] Update `docker-manager-lifecycle.test.ts` — api-proxy exits during startup
- [x] Update `docker-manager-lifecycle.test.ts` — api-proxy exits (1) near-duplicate
- [x] Update `docker-manager-lifecycle.test.ts` — squid health check failure
- [x] Update `docker-manager-lifecycle.test.ts` — squid exits during startup

- [x] Replace all remaining inline `upCalls` filter+assertion patterns with `expectComposeUpAttempts()`
- [x] All 49 tests pass, lint clean (0 errors)
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

This PR refactors the Docker startup retry unit tests by centralizing common mock setup and repeated “compose up attempt count” assertions into shared test helpers, reducing boilerplate across startup-retry scenarios.

Changes:
- Added `mockStartupRetry()` and `expectComposeUpAttempts()` helper utilities for startup-retry test cases.
- Updated `docker-manager-lifecycle.test.ts` to use the new helpers instead of inline mock sequences and repeated `mock.calls` filtering.

Copilot's findings

> [!TIP]
> <a>Add Copilot custom instructions for smarter, more guided reviews. Learn how to get started.

- **Files reviewed:** 2/2 changed files
- **Comments generated:** 1

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/docker-manager-lifecycle.test.ts:20</file>
<original_commit>63d6129</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
PR description says all remaining inline `mockExecaFn.mock.calls.filter(...includes('up'))` assertions were replaced, but there are still occurrences in other tests (e.g. `src/container-lifecycle-branches.test.ts` and `src/container-lifecycle-retry-timeout.test.ts`). Either update the PR description to scope this refactor to `docker-manager-lifecycle.test.ts`, or apply `expectComposeUpAttempts()` to the remaining files for consistency.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@lpcox</author>
@copilot address the review feedback and fix the pr title and summary
</comment_old>

<comment_new>
<comment_id>4755994936</comment_id>
<author>@lpcox</author>
@copilot address the review feedback and fix the pr title and summary
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>

```
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. Treat <comment_old>
comments as context only and do not make changes solely to
satisfy them.
* Only act on <comment_new> comments that explicitly mention you
(@copilot). To understand the context of comments that
explicitly mention you, you may use other comments that do not
explicitly mention you. Do not act on comments that don't
explicitly mention you or reply to them, even if they are marked
as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion
in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or
a suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```

<prompt>

@copilot address the review feedback and fix the pr title and summary

<prev_assistant>

Session paused: a tool permission request was not answered in time. Approve the tool in the conversation to resume from this point.

<current>

<prompt>

Allow "Create or update issue/pull request"